

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..

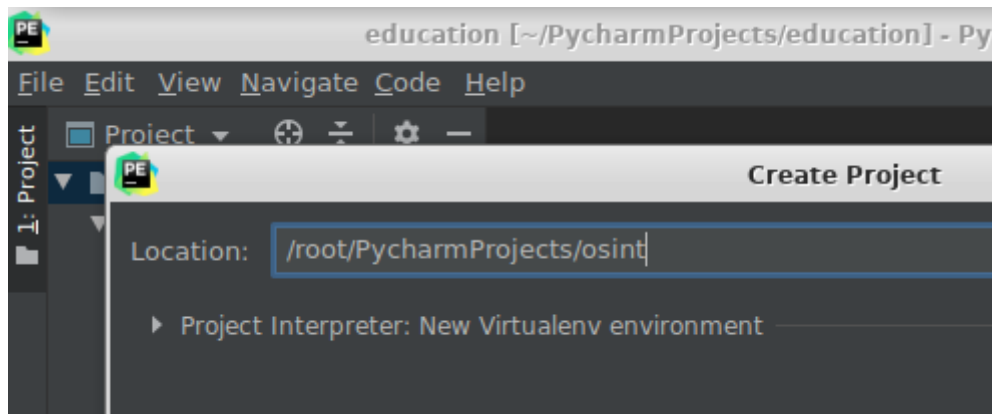


-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

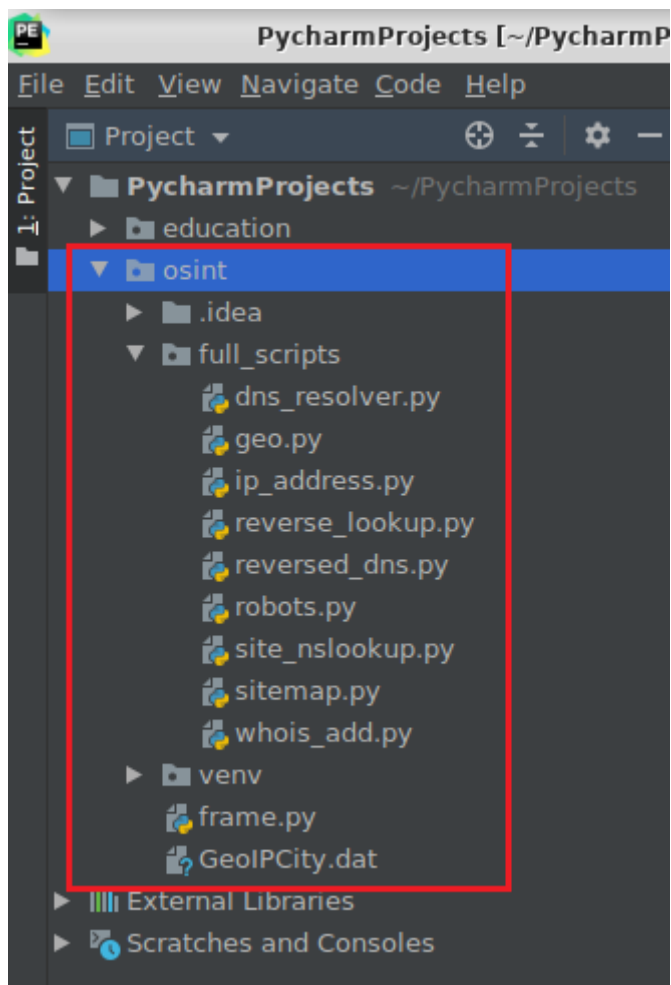
На прошлом занятии мы создали мини-фреймворк **OSINT BOX**. Во время его создания приходилось устанавливать ряд модулей. И чтобы конечному пользователю не приходилось выискивать и устанавливать необходимые модули, нужно создать файл, который будет содержать все требуемые зависимости. Тогда установка модулей сведётся к одной команде в терминале: `pip install -r requirements.txt` или `pip3 install -r requirements.txt`

Чтобы успешно создать файл, сделайте новый проект:

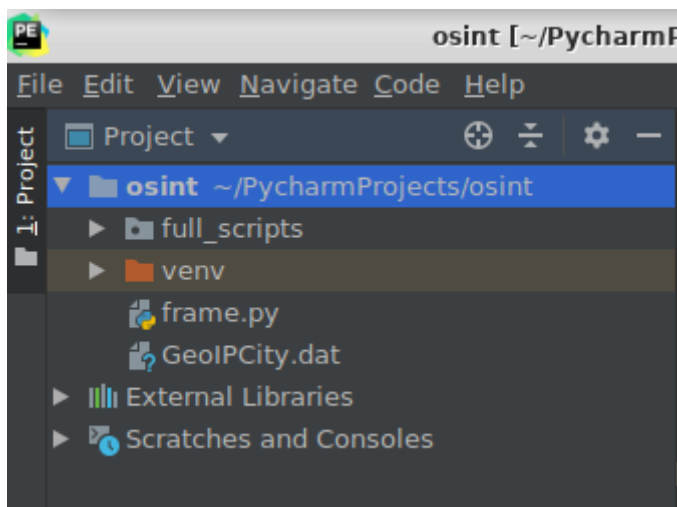
File ==> New Project... ==> имя проекта



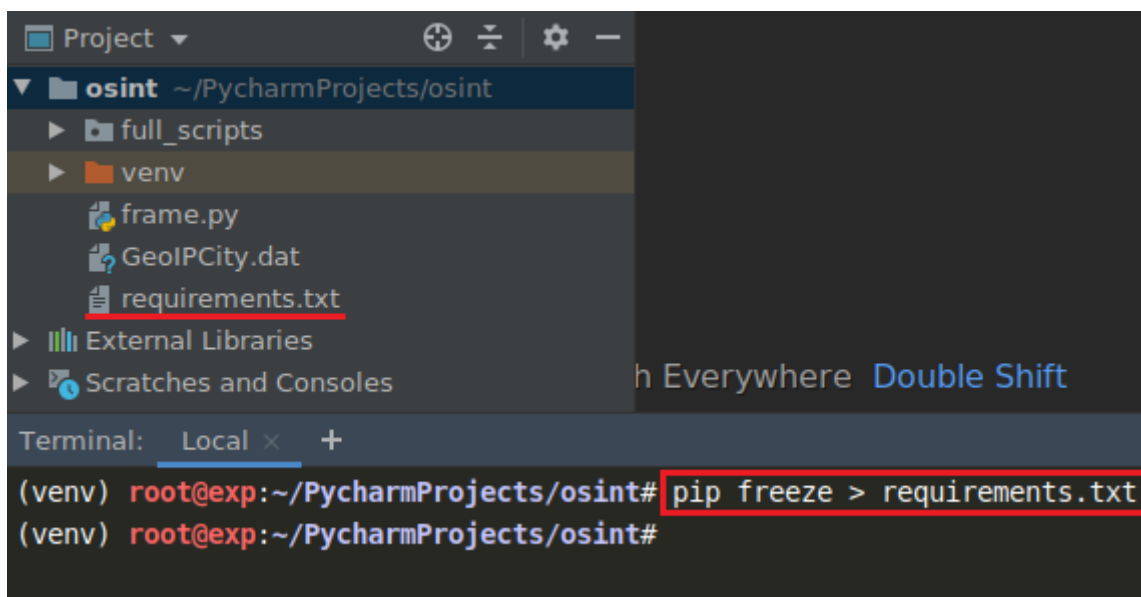
В созданный проект скопируем все наши файлы фреймворка. Должна получиться приблизительно такая структура:



Так как при копировании у нас были открыты оба проекта, то предварительно переоткроем только нужный нам проект osint **File ==> Open ==> osint**



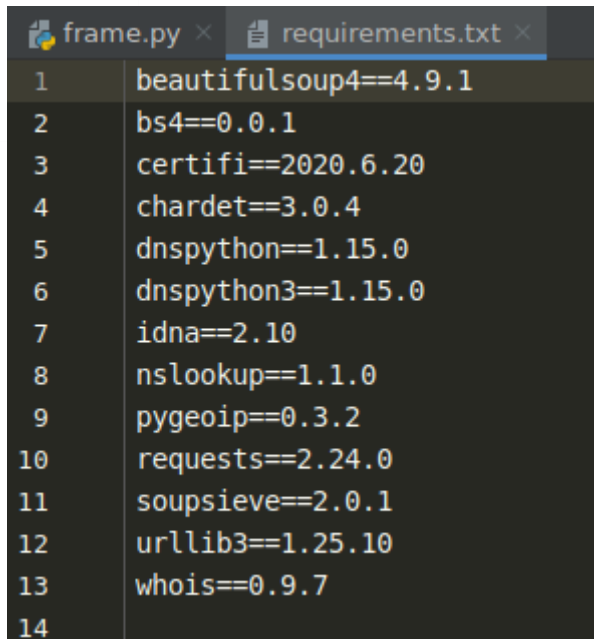
Далее в терминале вводим команду **pip freeze > requirements.txt** и в корне проекта у нас появится файл **requirements.txt**



Однако открыв requirements.txt вы обнаружите что файл пуст. Так как под каждый проект создаётся виртуальная среда, то никаких установленных модулей там не будет кроме pip и setuptools (они не будут в requirements.txt). Поэтому придётся всё устанавливать заново.

Хорошая практика это создавать программы, которые состоят из множества модулей, всегда в новом проекте, тогда не будет никакой путаницы и лишних манипуляций.

После установки нужных модулей, повторим команду в терминале **pip freeze > requirements.txt**. Теперь файл действительно содержит перечень всех необходимых зависимостей. Если в процессе создания фреймворка вы ставили какие-то пакеты, от использования которых вы в итоге отказались, то нужно их сначала удалить, чтобы они не попали в requirements.txt.



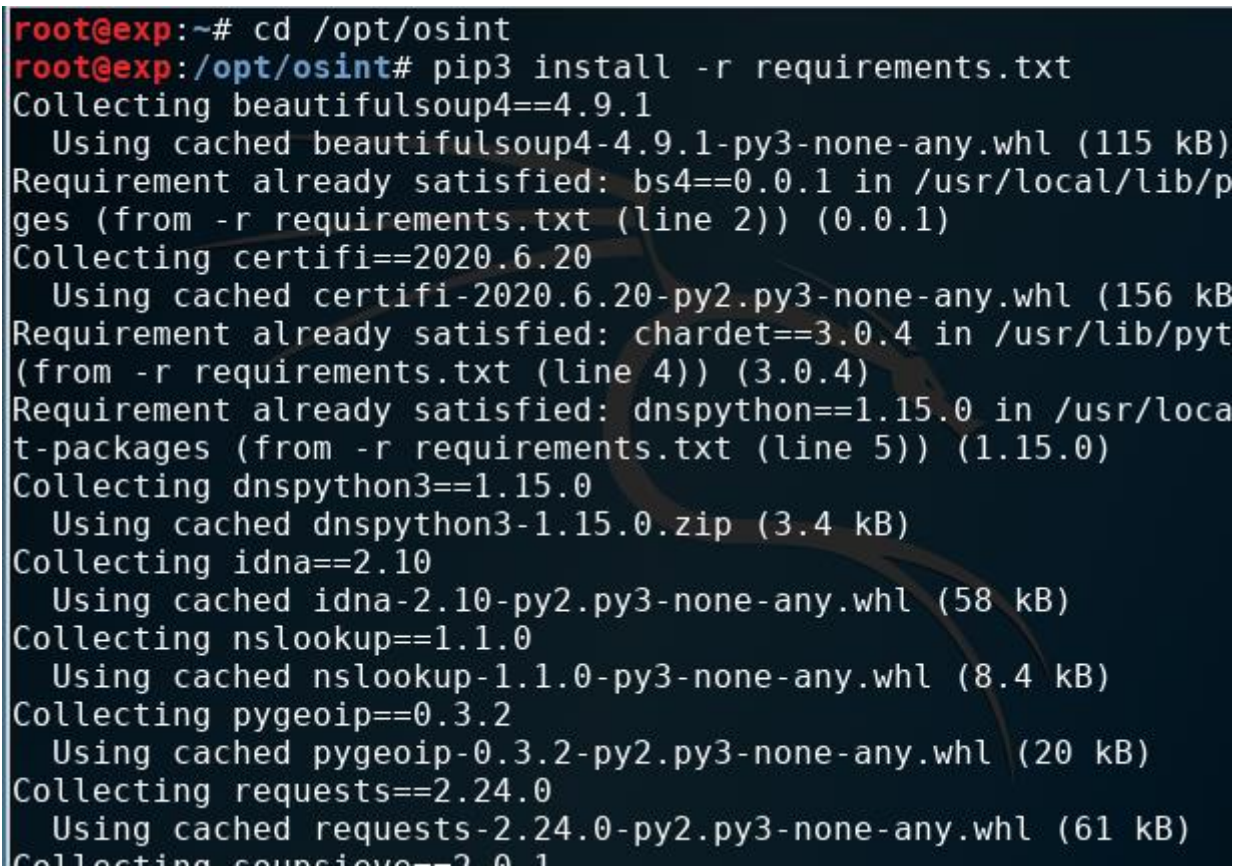
```

frame.py x requirements.txt x
1 beautifulsoup4==4.9.1
2 bs4==0.0.1
3 certifi==2020.6.20
4 chardet==3.0.4
5 dnspython==1.15.0
6 dnspython3==1.15.0
7 idna==2.10
8 nslookup==1.1.0
9 pygeoip==0.3.2
10 requests==2.24.0
11 soupsieve==2.0.1
12 urllib3==1.25.10
13 whois==0.9.7
14

```

Последний штрих – создадим в корне проекта файл **readme.txt** и напишем туда сообщение **Use the command to install dependencies "pip install -r requirements.txt" or "pip3 install -r requirements.txt"**.

Теперь проверим как это работает. Предположим юзер скачал наш код в zip с Git, или просто получил архив по любому каналу. Распаковал его в какую-нибудь папку, например **opt**. Прочёл файл **readme.txt**, и для работы с фреймворком ввёл команду установки зависимостей **pip3 install -r requirements.txt**:



```

root@exp:~# cd /opt/osint
root@exp:/opt/osint# pip3 install -r requirements.txt
Collecting beautifulsoup4==4.9.1
  Using cached beautifulsoup4-4.9.1-py3-none-any.whl (115 kB)
Requirement already satisfied: bs4==0.0.1 in /usr/local/lib/p
ges (from -r requirements.txt (line 2)) (0.0.1)
Collecting certifi==2020.6.20
  Using cached certifi-2020.6.20-py2.py3-none-any.whl (156 kB)
Requirement already satisfied: chardet==3.0.4 in /usr/lib/pyt
(from -r requirements.txt (line 4)) (3.0.4)
Requirement already satisfied: dnspython==1.15.0 in /usr/loca
t-packages (from -r requirements.txt (line 5)) (1.15.0)
Collecting dnspython3==1.15.0
  Using cached dnspython3-1.15.0.zip (3.4 kB)
Collecting idna==2.10
  Using cached idna-2.10-py2.py3-none-any.whl (58 kB)
Collecting nslookup==1.1.0
  Using cached nslookup-1.1.0-py3-none-any.whl (8.4 kB)
Collecting pygeoip==0.3.2
  Using cached pygeoip-0.3.2-py2.py3-none-any.whl (20 kB)
Collecting requests==2.24.0
  Using cached requests-2.24.0-py2.py3-none-any.whl (61 kB)
Collecting soupsieve==2.0.1

```

Всё прошло успешно, никаких ошибок не возникло.

Осталось запустить программу.



```

Файл  Правка  Вид  Терминал  Вкладки  Справка
root@exp:/opt/osint# python3 frame.py

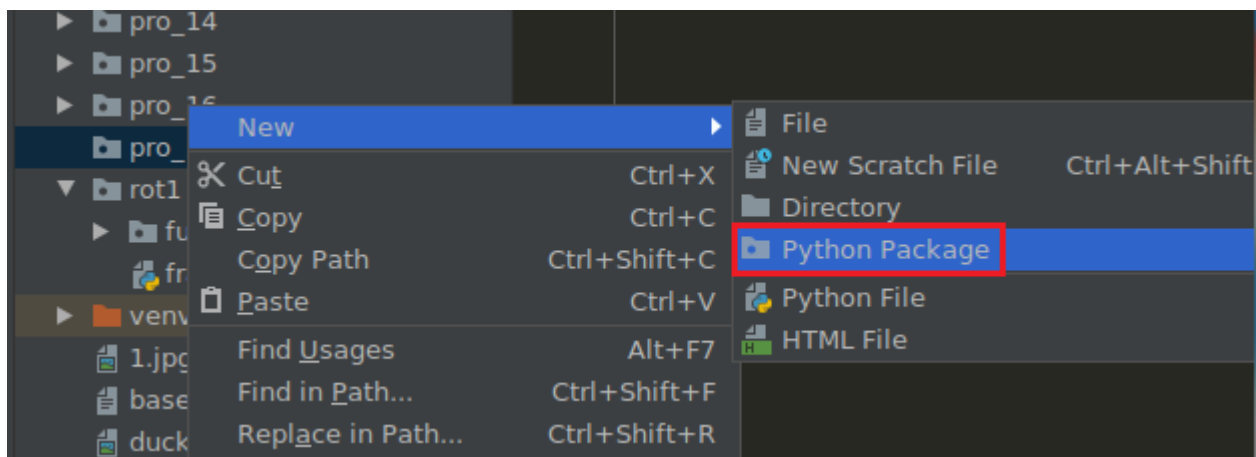
OSINT BOX

0. Exit the program
1. Host IP
2. Site location
3. Whois
4. Nslookup
5. DNS MX-Record
6. Reverse DNS
7. robots.txt
8. sitemap.xml
9. Reverse ip lookup

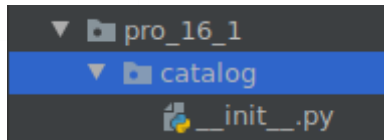
Enter the option number: 1
-----
1. Host IP
-----
Enter host: ctftime.org
IP Address of ctftime.org is 109.233.56.78

Enter the option number: 0
  
```

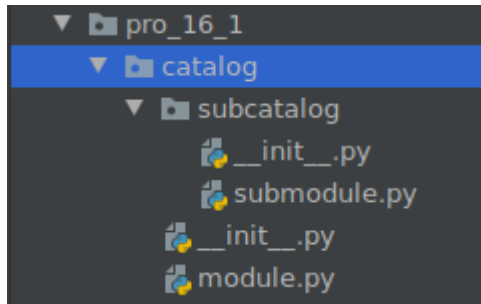
Теперь будем разбираться с более сложной структурой на простейших примерах. Создадим пакет **New ==> Python Package ==> имя пакета**.



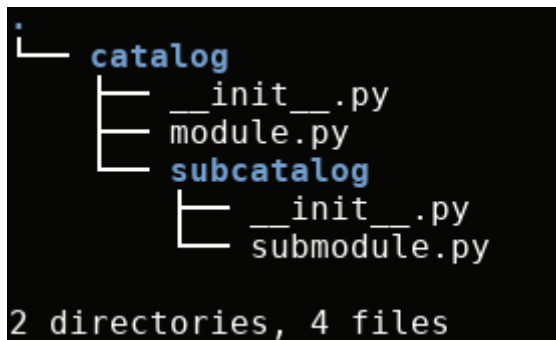
При создании пакета автоматически также создаётся файл `__init__.py`, и наличие этого файла указывает что у нас не просто папка, а именно пакет.



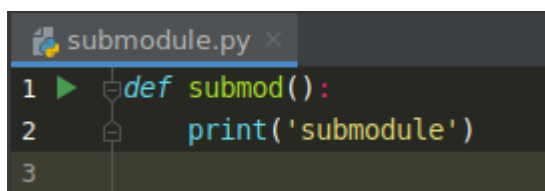
Теперь в пакет добавим файл `module.py` а также ещё один пакет `subcatalog`, в котором также создадим файл `submodule.py`



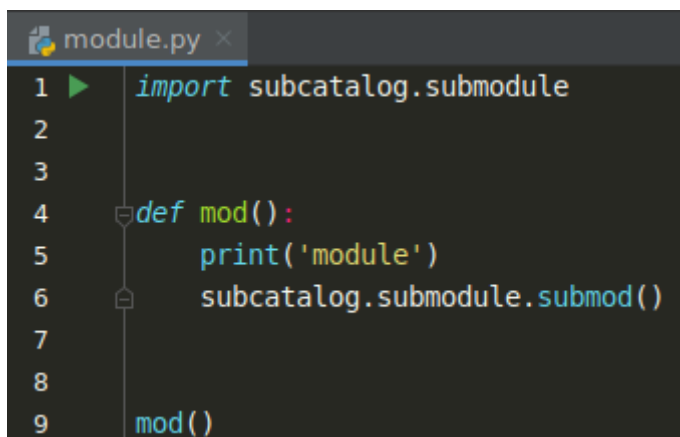
Соответствующая структура выглядит так:



В файл `submodule.py` просто напишем вывод сообщения `submodule`



И чтобы мы смогли запустить функцию `submod()` из `module.py`, необходимо сделать импорт модуля `submodule` через точку каталога `subcatalog`.



Или так:

```
from subcatalog import submodule

def mod():
    print('module')
    submodule.submod()

mod()
```

При запуске мы получим вывод из обоих файлов, находящихся в разных пакетах.

```
module x
module
submodule

Process finished with exit code 0
```

Когда интерпретатор Python импортирует каталог в первый раз, он автоматически запускает программный код файла `__init__.py` этого каталога.

При импортировании вложенного модуля, всегда сначала импортируются модули инициализации всех родительских пакетов, то есть находящихся выше.

Поэтому также будет работать такой вариант импорта:

```
from pro_16_1.catalog.subcatalog.submodule import submod

def mod():
    print('module')
    submod()

mod()
```

Теперь сделаем файл **rootmodule.py** в корневой папке, получится следующая структура:

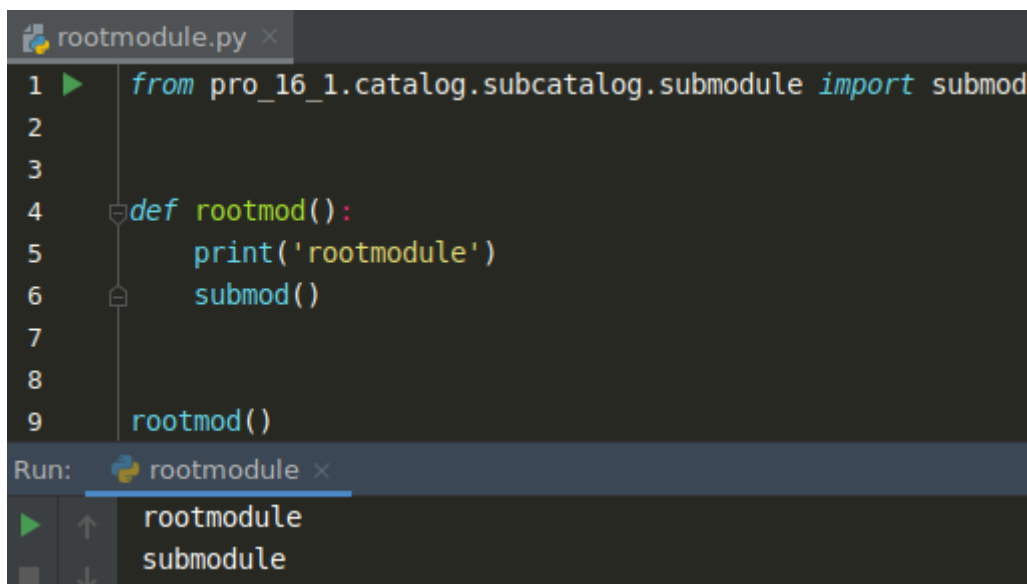
```

.
├── catalog
│   ├── __init__.py
│   ├── module.py
│   ├── pycache
│   │   ├── __init__.cpython-37.pyc
│   │   └── module.cpython-37.pyc
│   └── subcatalog
│       ├── __init__.py
│       ├── pycache
│       │   ├── __init__.cpython-37.pyc
│       │   └── submodule.cpython-37.pyc
│       └── submodule.py
└── rootmodule.py

```

4 directories, 9 files

И если мы хотим запустить функцию `submod()` из `rootmodule.py`, то импортирование делается таким же образом, через перечисление всех директорий.



```

rootmodule.py x
1  from pro_16_1.catalog.subcatalog.submodule import submod
2
3
4  def rootmod():
5      print('rootmodule')
6      submod()
7
8
9  rootmod()

```

Run: rootmodule x

```

rootmodule
submodule

```

Однако если мы попробуем импортировать функцию `mod()` из `rootmodule.py`, то у нас ничего не выйдет.

Подсистема интерпретатора, отвечающая за загрузку модулей, кэширует уже загруженные пакеты и модули. Каждый конкретный модуль загружается только один раз, в том числе и инициализирующие модули `__init__.py`. То есть подключение «по цепочке» вниз у нас не получится, вылетит ошибка.


```

rootmodule.py x
1  from pro_16_1.catalog.module import mod
2
3
4  def rootmod():
5      print('rootmodule')
6      mod()
7
8
9  rootmod()

Run: rootmodule x
Traceback (most recent call last):
  File "/root/PycharmProjects/education/pro_16_1/rootmodule.py", line 1, in <module>
    from pro_16_1.catalog.module import mod
  File "/root/PycharmProjects/education/pro_16_1/catalog/module.py", line 1, in <module>
    import subcatalog.submodule
ModuleNotFoundError: No module named 'subcatalog'

```

Импорт будет удачным только если в файле `module.py` не будет ещё одного импорта вниз по дереву. Поэтому эти моменты нужно учитывать при создании иерархии фреймворка.

Но если нам очень хочется получить результаты работы такие как в функции `mod()` из `rootmodule.py`, то есть обходной путь. Можно в инициализирующем модуле написать всё что угодно – функции, переменные и т.д.

```

Project
└─ pro_16_1
   └─ catalog
      └─ subcatalog
         └─ __init__.py
            └─ module.py
               └─ rootmodule.py

__init__.py x
1  print('module')
2

```

И при запуске программы `rootmodule.py` это будет выполнено. Обратите внимание, что инициализирующий модуль срабатывает первым!

```

rootmodule x
module
rootmodule
submodule

Process finished with exit code 0

```

Исходя из вышесказанного, лучше не делать вложенность более одного уровня. Правильнее пакеты располагать рядом. Большая вложенность увеличивает цепочку, и заставляет искать костыльные решения.

Создание скрипта установки пакетов

Все варианты импорта выше были привязаны к какому-либо файлу. Рассмотрим теперь как можно установить пакет в систему, и импортировать потом модули в любой файл. То есть мы сделаем глобальную установку в **site-packages** - хранилище пакетов, так как это делается при установке пакетов через pip.

Вернёмся к нашему фреймворку, и переместим файл **GeoIPCity.dat** в папку **full_scripts**. Так как путь к файлу **GeoIPCity.dat** на разных машинах не будет одинаковым, то перепишем скрипт так, чтобы он сам определял корректный путь.

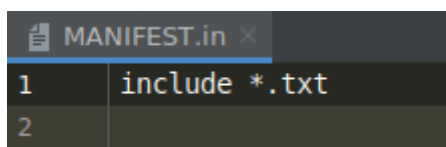
```
geo.py x
1  import pygeoip
2      from inspect import getsourcefile
3  from os.path import abspath
4
5
6  def geo_ip():
7      p = abspath(getsourcefile(lambda:0))
8      ip = input('Enter ip: ') # 109.233.56.78
9      gi = pygeoip.GeoIP(str(p)[:6] + 'GeoIPCity.dat')
10     city = gi.record_by_addr(ip)
11     for key in city:
12         if city[key] is None or city[key] == 0:
13             continue
14         else:
15             print(key, city[key], sep=": ")
```

И ещё добавим в папку **full_scripts** пустой файл **__init__.py** чтобы у нас получился пакет.

Для сборки скриптов нам понадобится создать файл **setup.py** в корне проекта. Внутри файла содержатся интуитивно понятные данные, и в параметре **packages** указывается пакет, который требуется собрать.

```
setup.py x  geo.py x
1  from distutils.core import setup
2
3  setup(
4      name='osint_box',
5      version='1.0',
6      description='OSINT module',
7      author='Some author',
8      author_email='example@example.com',
9      packages=['full_scripts'],
10     package_dir={'full_scripts': 'full_scripts'},
11     package_data={'full_scripts': ['*.dat']},
12 )
```

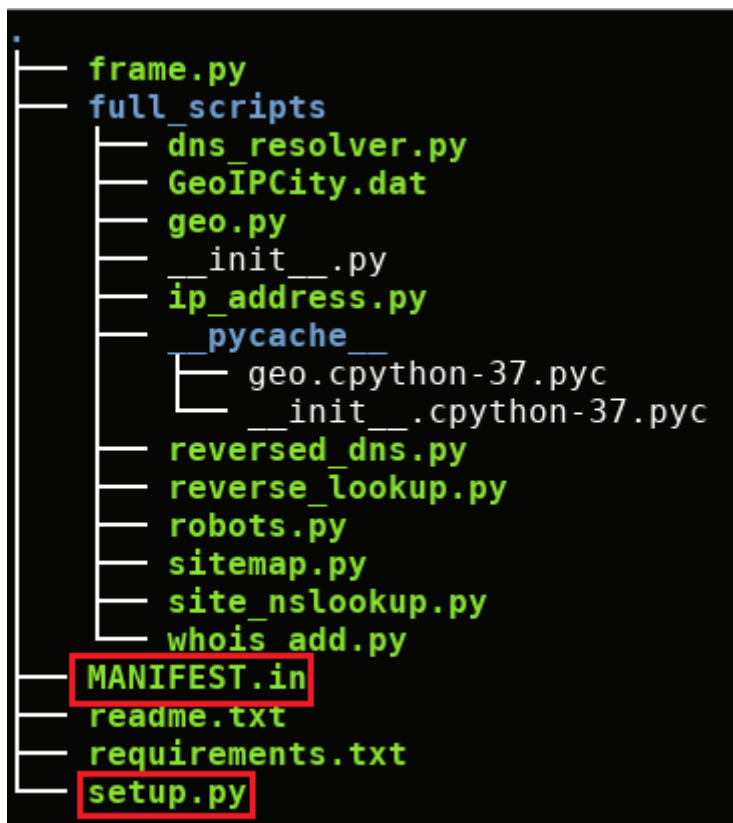
Теперь создадим в корне файл **MANIFEST.in** со следующим содержимым:



```
MANIFEST.in x
1 include *.txt
2
```

Таким образом в сборку попадут все файлы txt, а не только скрипты пайтона. В архив не попадёт файл frame.py, он нам не понадобится, и его можно удалить или оставить (никак не помешает процессу).

Итоговая структура должна получиться такая:

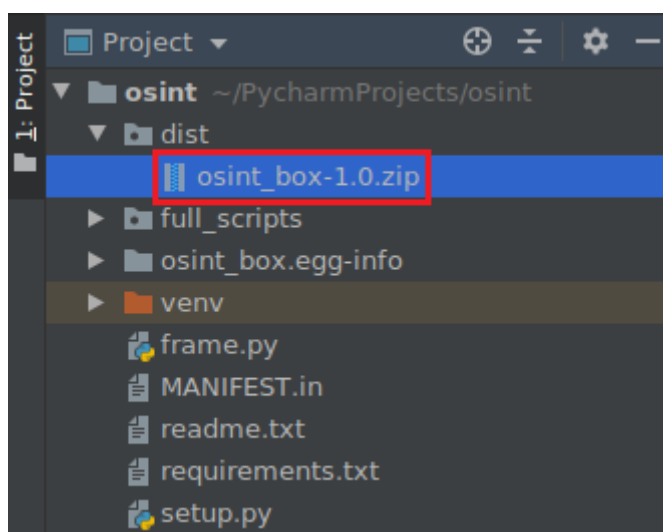


```
*
├── frame.py
├── full_scripts
│   ├── dns_resolver.py
│   ├── GeoIPCity.dat
│   ├── geo.py
│   ├── __init__.py
│   ├── ip_address.py
│   ├── pycache
│   │   ├── geo.cpython-37.pyc
│   │   └── __init__.cpython-37.pyc
│   ├── reversed_dns.py
│   ├── reverse_lookup.py
│   ├── robots.py
│   ├── sitemap.py
│   ├── site_nslookup.py
│   └── whois_add.py
├── MANIFEST.in
├── readme.txt
├── requirements.txt
└── setup.py
```

Подготовка закончена. Теперь в терминале соберём установочный архив командой `python setup.py sdist --formats=zip`. По-умолчанию создаётся в Unix архив [tar.gz](#), поэтому если хотите такой тип архива, то просто опустите в команде эту часть `--formats=zip`.

```
Terminal: Local x +
(venv) root@exp:~/PycharmProjects/osint# python setup.py sdist --formats=zip
running sdist
running egg_info
creating osint_box.egg-info
writing osint_box.egg-info/PKG-INFO
writing dependency_links to osint_box.egg-info/dependency_links.txt
writing top-level names to osint_box.egg-info/top_level.txt
writing manifest file 'osint_box.egg-info/SOURCES.txt'
reading manifest file 'osint_box.egg-info/SOURCES.txt'
reading manifest template 'MANIFEST.in'
writing manifest file 'osint_box.egg-info/SOURCES.txt'
```

Архив успешно создан, и находится в папке **dist**.



Распакуем архив прямо на месте.

```
Terminal: Local x +
(venv) root@exp:~/PycharmProjects/osint/dist# unzip osint_box-1.0.zip
Archive:  osint_box-1.0.zip
  creating: osint_box-1.0/
  creating: osint_box-1.0/full_scripts/
  creating: osint_box-1.0/osint_box.egg-info/
 inflating: osint_box-1.0/setup.cfg
 inflating: osint_box-1.0/PKG-INFO
 inflating: osint_box-1.0/setup.py
 inflating: osint_box-1.0/requirements.txt
 inflating: osint_box-1.0/MANIFEST.in
 inflating: osint_box-1.0/readme.txt
 inflating: osint_box-1.0/full_scripts/reversed_dns.py
```

Остаётся установить пакет в систему командой **python setup.py install**

```

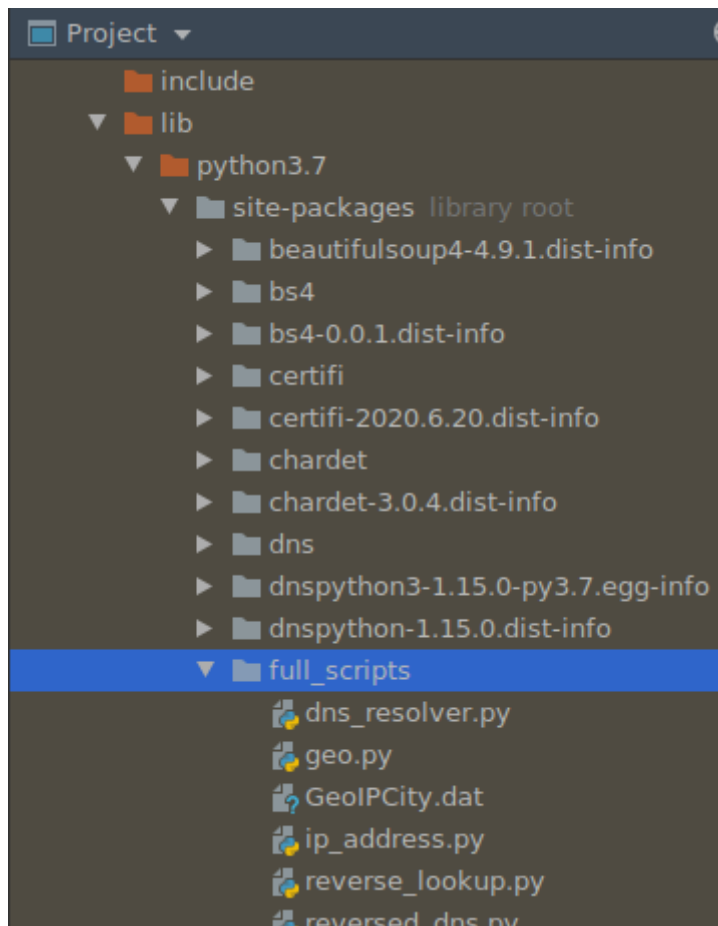
Terminal: Local x +
(venv) root@exp:~/PycharmProjects/osint/dist/osint_box-1.0# python setup.py install
running install
running bdist_egg
running egg_info
writing osint_box.egg-info/PKG-INFO
writing dependency_links to osint_box.egg-info/dependency_links.txt
writing top-level names to osint_box.egg-info/top_level.txt
reading manifest file 'osint_box.egg-info/SOURCES.txt'
reading manifest template 'MANIFEST.in'
writing manifest file 'osint_box.egg-info/SOURCES.txt'
installing library code to build/bdist.linux-x86_64/egg
running install_lib

```

Пакет установился по следующему пути:

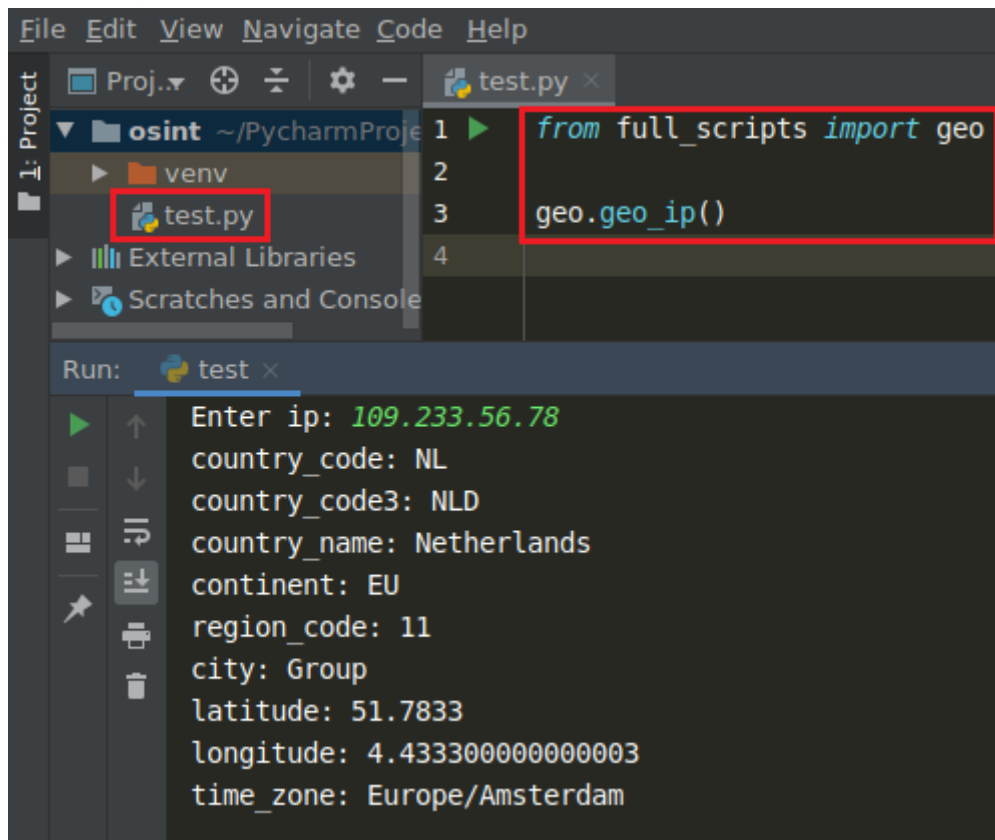
[/root/PycharmProjects/osint/venv/lib/python3.7/site-packages/full_scripts](#)

Внутри скрываются наши модули и база.



Теперь импорт полноценно должен работать. И мы можем любой интересующий нас модуль использовать в любой программе.

Для чистоты эксперимента удалим полностью всё что у нас есть в проекте, и создадим тестовый файл [test.py](#). Импортируем модуль с геоданными, и запустим. Всё работает.



The image shows a PyCharm IDE window. The top menu bar includes File, Edit, View, Navigate, Code, and Help. The left sidebar shows a project tree with a folder named 'osint' containing a 'venv' directory and a file named 'test.py'. The 'test.py' file is selected and highlighted with a red box. The main editor area shows the code in 'test.py':

```
1 from full_scripts import geo
2
3 geo.geo_ip()
4
```

The code is also highlighted with a red box. Below the editor, the 'Run' tab is active, showing the output of the script:

```
Run: test x
Enter ip: 109.233.56.78
country_code: NL
country_code3: NLD
country_name: Netherlands
continent: EU
region_code: 11
city: Group
latitude: 51.7833
longitude: 4.4333000000000003
time_zone: Europe/Amsterdam
```